

write-paper-guide

A User's Manual

A reading companion to one mathematical paper.

One paper, one reader, one guide.

Last revised: September 11, 2026

torsor lab

torsor.org

Contents

Preface: The Paper and You	iii
1 Two Parts, One Reader	1
The reader comes first	1
Part I — the overview	2
Part II — the walk-through	3
The appendix	3
What it is not	4
2 Running It	5
Starting	5
The reader interview	6
Where it lands	6
The outline	6
Writing, with checkpoints	7
Running it without checkpoints	7
3 What Comes Back	9
The directory	10
Building	11
Verification	12
When the math doesn't survive	12

Editing it afterwards	13
A Quick Reference	15
Invocation	15
Make targets	16
Files in a guide directory	16
Structure	17
LaTeX conventions	17
Voices	18
Words the house does not use	18
When the math doesn't render	19
What has to be installed	19

Preface: The Paper and You

Let's be honest with each other. There is a paper you need to understand, and reading it again is not working.

Maybe it's forty pages and the part you actually need is Section 6, which rests on Section 4, which assumes a construction the authors describe as standard. Maybe you're refereeing it in three weeks. Maybe a student is presenting it in your seminar and you would rather not meet the hard parts at the same moment they do. Whatever brought you here, the paper is not the problem. The paper is fine. The problem is the distance between what it assumes and what you have.

That distance is specific to you. A colleague two doors down would need a different paragraph explained, and would skim the one you need most. This is why “go read the survey first” so often fails — a survey is written for nobody in particular, and you are somebody in particular.

`write-paper-guide` produces a reading companion to one paper, calibrated to one reader. The output is a book in the same design as this one: PDF, HTML, EPUB, and Markdown, built from LaTeX source you can edit. It comes in two parts. Part I orients you — what the paper is about, why anyone wanted it, why it is hard, what is new in it. Part II walks the paper itself, following its own sections and its

own numbering, saying what carries each argument and where the difficulty actually lives.

It writes about the paper in the third person, always. The authors prove; Section 3 establishes; Theorem 4.2 states. A guide is not a contribution and must never read like one. It explains a paper that already exists, and every sentence in it should be checkable against that paper.

Three chapters follow. Chapter 1 is the shape of the thing: two parts, one reader, and why the calibration step is the one that decides whether the guide is any good. Chapter 2 is how to run it — what it asks you, how to answer, and how to pre-answer everything when you would rather it not stop. Chapter 3 is what lands on disk, how to rebuild it, and what to do when the math does not survive the trip to HTML. The appendix collects the commands and conventions worth looking up rather than memorizing.

If there is a paper open on your desk right now, Chapter 2 is the one.

Two Parts, One Reader

A reading guide has two parts and one reader.

That's the whole model. Everything else in this manual is detail.

The reader comes first

Before any outline exists, the skill asks who the guide is for. Field and subfield, and how close that is to the paper's area. What they already know against what needs developing. Why they're reading — refereeing, building on it, teaching from it, a seminar, curiosity. How much depth they want, and how much length they'll tolerate.

Those answers are written to `latex/reader-profile.md` inside the guide directory, and they are the tie-breaker for every later decision about how much to explain. The file is not compiled into the book. It sits beside the source as the record of what the guide was calibrated to, which matters when you come back in six months and wonder why Chapter 1 spends four pages on a construction you find obvious.

The calibration is the step that decides whether the guide is worth anything. A guide written for a generic reader develops the prerequisites that reader happens not to need and skips the one that actually blocks them. Answering the interview carefully costs you five minutes and is repaid on the first page.

The reader profile describes what someone is familiar with, never how good they are. “Comfortable with derived categories, hasn’t had reason to use perverse sheaves” is a calibration. “Expert” and “beginner” are grades, and grades tell the guide nothing it can act on.

Part I — the overview

Part I answers six questions about the paper. The default is two chapters, but the grouping bends to fit the paper; what doesn’t bend is that all six get answered.

- **What it is about.** Usually the longest. The problem, the objects, the definitions a reader needs in hand, the lay of the land. This is where the prerequisites the profile says are missing get developed.
- **Why it is interesting.** Context, history, stakes. Who has wanted this, and what its existence changes.
- **Why it is plausible.** The intuition that it should be true — special cases, analogies, earlier partial results.
- **Why it is hard.** Where the difficulty actually lives, and why the approach you’d try first fails.
- **Why it is new.** What was missing from the prior literature, in specific terms rather than as praise.
- **Why the authors pulled it off.** The perspective, technique, or sustained effort that made the difference.

The failure mode of Part I is generic admiration, and it is worth naming because it writes itself. “The paper makes a significant contribution to a central problem” answers none of the six questions. “Pre-

vious results needed the base field to be algebraically closed, and the obstruction was the descent step in Section 4” answers two of them.

Part II — the walk-through

Part II follows the paper’s spine. A short roadmap chapter first: the paper’s structure at a glance, what depends on what, a reading order, and what to skim on a first pass. Then one chapter per major section or cluster of results, taking the paper in its own order.

Each of those chapters does four things. It says what that part of the paper accomplishes. It restates the results that matter, carrying the paper’s own numbering. It explains how the argument is built — which lemma is the engine and which are bookkeeping. And it says where the difficulty is, so you know what to read line by line and what to take on faith the first time through.

Restated results are set in theorem environments that print the paper’s own number:

```
\begin{paperthm}[Theorem 4.2, slightly  
informally]  
...  
\end{paperthm}
```

Carrying the number is not decoration. It is what lets you check the guide against the paper in ten seconds, which you should do the first time a restatement surprises you. A paraphrase is always marked as one.

The appendix

Last comes a notation and reference card: the paper’s notation collected in one place, and a table mapping the guide’s chapters to the

paper's sections. This is the part you search rather than read, and it is deliberately dense so the prose chapters don't have to be.

What it is not

A guide is not a survey. Context appears in service of this paper, not as a tour of the field.

A guide is not original work. If a sentence reads as a new result, it is in the wrong document — and if the guide fills a gap the paper left, it says that plainly rather than quietly closing it.

A guide is not a summary either. A summary gets shorter; a guide gets longer, because the compressed passages are exactly the ones worth expanding.

Running It

Point the skill at a paper and answer a few questions. The rest is a conversation with checkpoints.

Starting

Invoke it with whatever identifies the paper — a path to a PDF, an arXiv identifier or URL, or enough of a citation to find it:

```
/write-paper-guide ~/papers/descent.pdf  
/write-paper-guide arXiv:0000.00000  
/write-paper-guide the paper on heights of  
special points that everyone cites
```

Given only a citation, it finds the paper and confirms with you that it has the right one before investing in a full read. Confirm carefully. A guide to the wrong paper is expensive in a way that is not obvious until page thirty.

Then it reads the paper — properly, not the abstract — and does not start writing until it can state in one paragraph what the paper proves and how. If that paragraph looks wrong to you when it appears, say so immediately rather than waiting to see whether the guide recovers. It won't.

The reader interview

Next it asks about the reader, as a short batch of questions rather than one at a time. Answer them for whoever will actually read the guide. If that's you, say so and answer from your own position.

You can skip the interview by pointing at a document instead: notes you already have, or a profile file for this reader if you keep them. It reads that, tells you how it read it, and proceeds.

Two answers do more work than the rest. **Why they're reading** sets what counts as the point — a guide for a referee and a guide for a seminar speaker diverge almost immediately. **What they already know** decides how much of Part I is setup. Spend your effort on those two.

Where it lands

Then it proposes a location and waits. A paper sitting in its own folder usually gets a **guide/** subdirectory beside the PDF; a loose paper gets a new directory named for it, somewhere you choose.

If a guide directory is already there, it looks at what's in it and asks whether to extend it or start fresh. It doesn't overwrite anything without asking.

The outline

Next comes the proposed outline: the two parts, the chapters within them, and the appendix. This is the cheapest moment in the whole process to change your mind about scope, so read it properly. Merging two chapters here costs a sentence. Merging them after they're written costs a rewrite.

The voice is settled at the same time. The default is 01-direct, which is the register this manual is written in and the one the family is tuned for. There is a more digressive 02-wandering if you want it; ask for it by name at this point.

Writing, with checkpoints

Part I is written first, in full, and shown to you before Part II begins. That order is deliberate: Part II inherits Part I's framing and depth, so a correction made at the end of Part I is a correction to everything, and the same correction made later is not.

Part II then arrives a chapter at a time, each one shown to you before the next is started. The thing worth watching across those checkpoints is proof depth — how much of each argument gets reconstructed. It's the dial most likely to be set wrong at first, and the one that's cheapest to adjust early. Say “more detail on the proofs” or “less” and it carries forward.

The roadmap chapter is written early, since it frames the rest. The notation appendix is written last, once there's something to collect.

Running it without checkpoints

Sometimes you don't want a conversation. You want to come back to a finished guide. Pre-answer everything it would stop to ask, in one argument string:

```
/write-paper-guide The paper is at
~/papers/descent.pdf. Do not ask me
anything --
take every default and proceed. The reader
is an
active researcher in an adjacent field:
comfortable with the general area and its
```

```
standard machinery, familiar with the
  objects by
name, but not a specialist in this corner
  and
without the local folklore. They are
  reading in
order to referee it, so go to proof depth
  and
skip nothing -- where the paper compresses
  ,
reconstruct. Length: exhaustive, no budget
  .
Location: ~/papers/descent/guide/.
```

That covers the paper, the reader, the purpose, the depth, the length, and the location, which is everything the skill would otherwise pause on. This is exactly how `write-critical-guide` drives it as a subroutine, and it is the pattern to copy when you want an unattended run.

An unattended run gives up the checkpoints, and the checkpoints are where the depth gets corrected. If the depth comes out wrong, the fix is another full run, not an edit. Pre-answer when you're confident about the reader and the depth; keep the checkpoints when you're not.

What Comes Back

A guide directory is a self-contained book. You can move it, hand it to someone, or rebuild it on a machine that has never had the plugin installed.

The directory

```
guide/  
  Makefile  
  check-build.py          verification,  
    copied in  
  tex2torsor/            converter, copied  
    in  
  latex/  
    main.tex             preamble, title,  
  inputs  
  STYLE.md               the style guide in  
  force  
  reader-profile.md      the calibration  
  chapters/  
    00-preface.tex  
    01-what-its-about.tex  
    ...  
    99-notation-and-reference.tex  
  html/  
  epub/  
  markdown/              build output
```

The tools are copied in rather than linked, which is why the directory survives a plugin update or an uninstall. It also means each guide carries its own copy, so a fix to the converter reaches old guides only if you copy it over.

STYLE.md is assembled when the guide is scaffolded: the format mechanics for this genre, followed by the voice file, with a header recording which voice was used. It is the answer to “why is it written like this” and the thing to read before editing prose by hand.

Building

Four formats, four targets:

```
make pdf      latex/main.pdf, via latexd
make html     html/manual.html, via
               tex2tutor
make epub     epub/manual.epub, via pandoc
make md       markdown/manual.md, GFM
make check    verify the build
make view     build html, open it in lab-
               view
make clean    remove the build output
```

The PDF is the format the design is tuned for and the one to trust. The others are conveniences: HTML to read on a screen or send a link to, EPUB for a tablet, Markdown for grep and for pasting into a message.

`latexd` exits 0 even when LaTeX has failed, so `make pdf && echo ok` can lie to you. `make check` is what catches it — it verifies the PDF exists, is newer than every source file, has a sane page count, and that the log carries no errors or undefined references. Run it. A one-page PDF where you expected forty is the usual shape of the failure.

Verification

`latexd` compiles in a temporary directory and does not keep the LaTeX log, so after a `latexd` build `make check` has no log to read and fails on that alone. Build once with `latexmk` and the log lands in `latex/`, where the checker looks:

```
latexmk -pdf -cd latex/main.tex
```

Or point the checker at a log elsewhere with its `--log` option.

`make check` is the mechanical tier. It reports LaTeX errors, undefined references and citations, overfull boxes with their overrun in points, non-ASCII characters inside listings, and missing EPUB or Markdown output. Exit 0 with warnings is possible, so read the report rather than the exit code.

Above that sits the publication pass, which is the gate the whole family shares. It runs the mechanical checks, then inspects a targeted sample of pages by eye — title page, colophon, one page per chapter, every page a mechanical check flagged, and the display math — and returns an evidence report. Not “everything looks good”: page counts, specific findings with page numbers, and an explicit list of what was accepted as an exception. The guide is not finished until that report is clean.

When the math doesn’t survive

The PDF renders everything. The HTML and EPUB paths are weaker, and they fail quietly, which is worse than failing loudly.

The converter cannot handle over-accented symbols such as `\widetilde` and `\overline`, `\frac`, extensible arrows, or `array` and `aligned` — which means commutative diagrams. What you get instead is raw TeX sitting in the page where an equation should be.

Three fixes, applied to the guide and not to the shared converter:

- **EPUB:** add `--mathml` to the `epub` target's `pandoc` call. Native MathML, and it renders properly.
- **HTML:** drop `inject-mathjax.py` at the guide root and add it as a post-step to the `html` target. MathJax renders exactly the fallback spans the converter leaves behind. The script is idempotent and has no build-time dependencies; the full text of it is in the `family lessons` file.
- **Notation:** where even that would be awkward, write a renderable equivalent in the source, and note the paper's own glyph once in prose and once in the appendix — a plain $\operatorname{colimit}$ operator in place of an extensible one, an isomorphism sign in place of an arrow with a tilde over it. All three formats then agree, at the cost of restating the paper's exact glyphs once.

The other thing that needs telling: if the guide uses `pitfallbox` — the callout for the subtlety that actually trips people — the converter drops it in HTML unless you pass it a mappings file that says how to render it. The mapping is four lines of YAML and lives in the `family lessons`.

Editing it afterwards

It's LaTeX. Edit the chapter files and rebuild. Two habits worth keeping: don't touch the palette, fonts, or box definitions in the preamble, since those are what make the family look like a family; and re-run `make check` after editing, because the failure you just introduced is the quiet kind.

If you want more than an edit — a chapter added, the depth changed throughout — hand the guide back to the skill and say what should

be different. It reads what's there and works alongside it rather than starting over.

Quick Reference

Everything worth looking up rather than remembering.

Invocation

```
/write-paper-guide <pdf path>  
/write-paper-guide arXiv:<id>  
/write-paper-guide <citation or  
description>
```

Unattended template — pre-answers every question the skill would stop on:

```
The paper is at <path>. Do not ask me  
anything --  
take every default and proceed. The reader  
is  
<field, and how close to the paper's area  
>: <what  
they know>, but <what they lack>. They are  
reading in order to <purpose>, so <depth  
instruction>. Length: <budget or "  
exhaustive">.  
Location: <path>/guide/.
```

Make targets

Target	What it does
pdf	latex/main.pdf via latexd, falling back to latexmk
html	html/manual.html via tex2torsk
epub	epub/manual.epub via pandoc
md	markdown/manual.md, GitHub-Flavored Markdown
check	run check-build.py: log scan, freshness, format checks
view	build HTML and open it in lab-view
clean	remove html/, epub/, markdown/

Files in a guide directory

Path	What it is
latex/main.tex	preamble, title page, chapter inputs
latex/STYLE.md	base mechanics + voice; records the voice
latex/reader-profile.md	the calibration; not compiled
latex/chapters/	one .tex per chapter
check-build.py	verification, copied in
tex2torsk/	HTML converter, copied in
html/, epub/	build output; gitignored
markdown/	build output; gitignored

Structure

Part	Contents
Preface	what the paper is, how to use the guide
Part I	the six questions, usually in two chapters
Part II	roadmap, then one chapter per major section of the paper
Appendix	notation card, plus chapter-to-section map

The six questions of Part I: what it is about; why it is interesting; why it is plausible; why it is hard; why it is new; why the authors were able to pull it off.

LaTeX conventions

Macro or environment	Use
<code>\code{}</code>	inline command, file, field, or flag name
<code>lstlisting</code>	multi-line code; ASCII only
<code>notebox</code>	something a reader might genuinely miss
<code>warnbox</code>	real harm: data loss, a build that lies
<code>pitfallbox</code>	the subtlety that actually trips readers
<code>paperthm</code>	restate a theorem, carrying the paper's number
<code>paperdefn</code>	likewise, a definition
<code>paperlem</code>	likewise, a lemma
<code>paperprop</code>	likewise, a proposition

Restatements always carry the paper's own number in the optional argument:

```
\begin{paperthm}[Theorem 4.2, slightly
informally]
```

Voices

Voice	Register
01-direct	the default and the proven one: direct, dry, leads with the point
02-wandering	experimental; digressive, several lines of thought braided

Ask for a non-default voice when the outline is confirmed. The choice is recorded in `reader-profile.md` and in the `STYLE.md` header.

Words the house does not use

Utilize (use “use”), *leverage*, *seamless*, *robust*, *powerful*, *simple*, *easy*, *straightforward*, *clean*, *load-bearing*. Also *we prove* and *our result*, which belong to the paper and not to a guide; *elegant* and *beautiful* as filler; and any throat-clearing that opens a section without saying anything.

Numbers: spell out one through nine, numerals for 10 and above, and always numerals for theorem, section, equation, page, and reference numbers.

When the math doesn't render

Symptom	Fix
Epub shows raw TeX	add <code>--mathml</code> to the epub pandoc call
HTML shows raw TeX	add <code>inject-mathjax.py</code> as a post-step to html
Diagrams and over-accented lost	write renderable notation; note the paper's glyph in prose
pitfallbox gone in HTML	pass a mappings YAML to the converter with <code>-c</code>
PDF is one page, build "succeeded"	<code>latexd</code> lied; read <code>latex/main.log</code>
<code>make check</code> finds no log	<code>latexd</code> discards it; build once with <code>latexmk</code>

What has to be installed

Tool	Needed for
<code>pdflatex</code>	the PDF; with <code>ebgaramond</code> , <code>cabin</code> , <code>microtype</code> , <code>mdframed</code> , <code>titlesec</code> , <code>fancyhdr</code> , <code>listings</code>
<code>latexmk</code>	driving the PDF build (<code>latexd</code> wraps it)
<code>pandoc</code>	HTML, EPUB, and Markdown
<code>python3</code>	the converter and <code>check-build.py</code>
<code>lab-view</code>	optional; only <code>make view</code> uses it